

Tugas 2

Sistem Paralel dan
Terdistribusi - A

Distributed Synchronization System



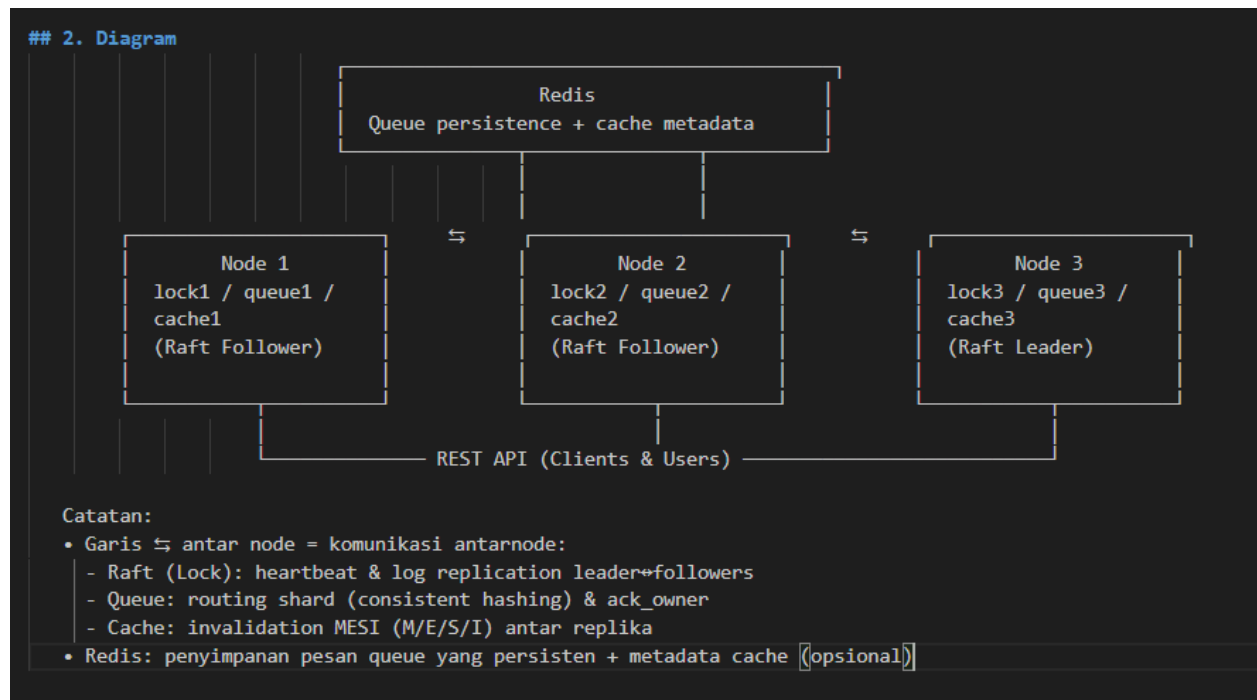
Disusun Oleh :

Muhammad Zaki Afriza 11231067

28 Oktober 2025

A. Technical Documentation

Arsitektur Sistem



Lock Manager (Raft, 3 node): menangani shared/exclusive locks; write hanya via leader; redirect follower $\rightarrow$ leader; deadlock detection (wait-for + timeout).

Distributed Queue (3 node): routing (topic,key) via consistent hashing $\rightarrow$ owners; persistence di Redis; consume mengubah status ke in-flight (visibility TTL) sampai ACK; at-least-once.

Distributed Cache (3 node): MESI (M/E/S/I) + invalidation; LRU eviction; TTL opsional; metrics (hits/misses/puts/evictions/inv_sent/inv_recv).

Redis: penyimpanan queue & metadata cache.

Penjelasan Algoritma

Raft (Lock Manager)

- Role: leader, follower, candidate; leader election via term & randomized election timeout.
- Log replication: setiap operasi lock (acquire/shared/exclusive, release) ditulis ke log; commit saat mayoritas (quorum) menyetujui.

- Safety: hanya leader yang terima write; follower redirect ({"detail":{"redirect":true}}).
- Partition handling: hanya partisi dengan quorum yang bisa commit; partisi minor menjadi follower/candidate.
- Deadlock detection: wait-for graph berbasis (resource → holders/waiters) + timeout; jika siklus terdeteksi atau timeout terlewati, request diputuskan (gagal) supaya sistem tidak hang.

Queue (Consistent Hashing + At-Least-Once)

- Sharding: hash ring atas (topic,key) → k owners (contoh 2); publish/consume diarahkan ke owner primer; fallback ke owner lain saat node gagal.
- At-least-once: consume(visibility_ttl) menandai pesan in-flight; bila tidak ACK sebelum TTL, pesan di-re-deliver.
- Persistence & recovery: pesan disimpan di Redis (list/set + status); node bisa recover dari crash tanpa kehilangan data.
- ACK & routing: ack langsung ke owner; ack_owner mengizinkan ACK via replica (proxy) dengan parameter owner.

Cache (MESI + LRU + TTL)

- State:
 - M (Modified): hanya satu node pemilik, nilai mutakhir; men-invalidate S/E di node lain.
 - E (Exclusive clean): satu pemilik, belum dimodifikasi; read ke S atau write ke M.
 - S (Shared): banyak node menyimpan salinan; write mengirim invalidation ke semua S → penulis menjadi M.
 - I (Invalid): tidak valid/terhapus/evicted.
- Replacement: LRU per node, kapasitas default (mis. 512 entries).
- TTL opsional untuk entri cache; expire → I.
- Metrics untuk observability: hits, misses, puts, evictions, invalidations (sent/recv).

API Documentation

(OpenAPI ringkas)

Catatan: Berikut ringkasan inti. Spesifikasi lengkap tersedia sebagai OpenAPI YAML (bisa dipisah di docs/api_spec.yaml).

openapi: 3.0.3 info:

```
title: Distributed Sync System API
version: "1.0.0"
servers:
  - url: http://localhost
paths:
  /readyz:
    get:
      summary: Health check
      responses: { "200": { description: OK } }

# ----- Lock (Raft) -----
/raft/leader:
  get:
    summary: Get Raft role/leader
    responses:
      "200": { description: JSON {role, leader} }

/lock/acquire:
  post:
    summary: Acquire lock (shared/exclusive)
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [resource, mode, client_id, timeout_ms]
            properties:
              resource: { type: string }
              mode: { type: string, enum: [shared, exclusive] }
              client_id: { type: string }
              timeout_ms: { type: integer }
    responses:
      "200": { description: Accepted (pending) atau Granted (token) }

/lock/release:
  post:
    summary: Release lock by token
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [resource, token]
            properties:
```

```
    resource: { type: string }
    token: { type: string }
responses:
  "200": { description: JSON {released: bool} }
```

/lock/state:

```
get:
  summary: Inspect lock state for a resource
parameters:
  - in: query
    name: resource
    required: true
    schema: { type: string }
responses:
  "200": { description: JSON {resource, mode, holders, queue} }
```

----- Queue -----

/queue/owners:

```
get:
  summary: Get owners of (topic,key)
parameters:
  - in: query; name: topic; required: true; schema: {type: string}
  - in: query; name: key; required: false; schema: {type: string}
responses: { "200": { description: JSON {owners, self} } }
```

/queue/publish:

```
post:
  summary: Publish message
parameters:
  - in: query; name: topic; required: true; schema: {type: string}
  - in: query; name: key; required: false; schema: {type: string}
requestBody:
  required: true
  content:
    application/json: { schema: { type: object } }
responses:
  "200": { description: JSON {msg_id, owners[]} }
```

/queue/consume:

```
post:
  summary: Consume messages (mark in-flight)
parameters:
  - in: query; name: topic; required: true; schema: {type: string}
  - in: query; name: key; required: false; schema: {type: string}
  - in: query; name: visibility_ttl; required: true; schema: {type: integer}
  - in: query; name: max; required: false; schema: {type: integer, default: 10}
```

responses:

"200": { description: JSON array of {msg_id, owner, payload} }

/queue/ack:

post:

summary: Acknowledge message at owner

parameters:

- in: query; name: topic; required: true; schema: {type: string}
- in: query; name: msg_id; required: true; schema: {type: string}

responses:

"200": { description: JSON {acked: bool} }

/queue/ack_owner:

post:

summary: Proxy ACK to a specific owner

parameters:

- in: query; name: topic; required: true; schema: {type: string}
- in: query; name: owner; required: true; schema: {type: string}
- in: query; name: msg_id; required: true; schema: {type: string}

responses:

"200": { description: JSON {acked: bool} }

----- Cache -----

/cache/put:

post:

summary: Put cache value (optional TTL)

parameters:

- in: query; name: ttl_ms; required: false; schema: {type: integer}

requestBody:

required: true

content:

application/json:

schema:

type: object

required: [key, value]

properties:

key: { type: string }

value: {}

responses:

"200": { description: JSON {stored: bool} }

/cache/get:

get:

summary: Get cache value

parameters:

- in: query; name: key; required: true; schema: {type: string}

```
responses:
  "200": { description: JSON {hit: bool, value?: any} }

/cache/state:
  get:
    summary: MESI state of a key
    parameters:
      - in: query; name: key; required: true; schema: {type: string}
    responses:
      "200": { description: JSON {key, state} }

/cache/metrics:
  get:
    summary: Cache metrics (hits/misses/puts/evictions/invalidation)
    responses:
      "200": { description: JSON metrics }
```

Deployment Guide & Troubleshooting

Prasyarat

- Docker & Docker Compose, Python 3.8+, Windows PowerShell (contoh perintah).
- Port: Locks 8080/8081/8082 · Queue 8180/8181/8182 · Cache 8280/8281/8282.

Menjalankan cluster

```
docker compose -f .\docker\docker-compose.yml up -d

# Health checks
curl.exe -s http://localhost:8080/readyz
curl.exe -s http://localhost:8180/readyz
curl.exe -s http://localhost:8280/readyz
```

Scaling

Tambah servis (replica) di docker-compose.yml atau jalankan node tambahan (port berbeda).
Queue & Cache otomatis mengenali peers dari env `*_PEERS`.

Konfigurasi (.env)

```
REDIS_URL=redis://redis:6379/0
LOCK_PEERS=http://lock1:8080,http://lock2:8081,http://lock3:8082
QUEUE_PEERS=http://queue1:8180,http://queue2:8181,http://queue3:8182
CACHE_PEERS=http://cache1:8280,http://cache2:8281,http://cache3:8282
```

```
LOG_LEVEL=INFO
```

Simulasi Network Partition (contoh)

```
$cid = docker compose -f .\docker\docker-compose.yml ps -q lock2
$net = (docker inspect -f "{{json .NetworkSettings.Networks}}" $cid |
ConvertFrom-Json).psobject.Properties.Name | Select-Object -First 1
docker network disconnect $net $cid # putuskan lock2
# amati /raft/leader & write → hanya quorum yang commit
docker network connect $net $cid # sambungkan lagi
```

Troubleshooting

- JSON decode error (PowerShell + curl.exe): gunakan Invoke-RestMethod atau --data-binary "@file.json".
- Lock acquire redirect: panggil /raft/leader, kirim write ke leader.
- Queue ACK gagal not_in_inflight: ACK harus ke owner yang memegang pesan (ack_owner), atau konsumsi ulang setelah TTL.
- Cache tidak langsung konsisten: tunggu $\leq 2s$ agar MESI invalidation menyebar; cek /cache/metrics.

B. Performance Analysis Report

Analysis Perform (Locust)

Konfigurasi uji:

- Jumlah user: 120
- Ramp-up (spawn rate): 30 user/detik
- Target host: cluster Docker lokal (lock/queue/cache + Redis)
- Skenario: campuran operasi cache:get/put, queue:publish/consume/ack_owner, dan lock:acquire_exclusive/release.

Hasil agregat (steady-state):

- RPS total: stabil di kisaran ~420–520 req/s (grafik “Total Requests per Second”).
- Average response time: ~150–200 ms.
- p95 latency: ~450–500 ms.
- Failure rate: 0% (tidak ada error pada tab Failures).
- Users aktif: 120 (sesuai ramp-up 30 → plateau di 120 pada grafik).

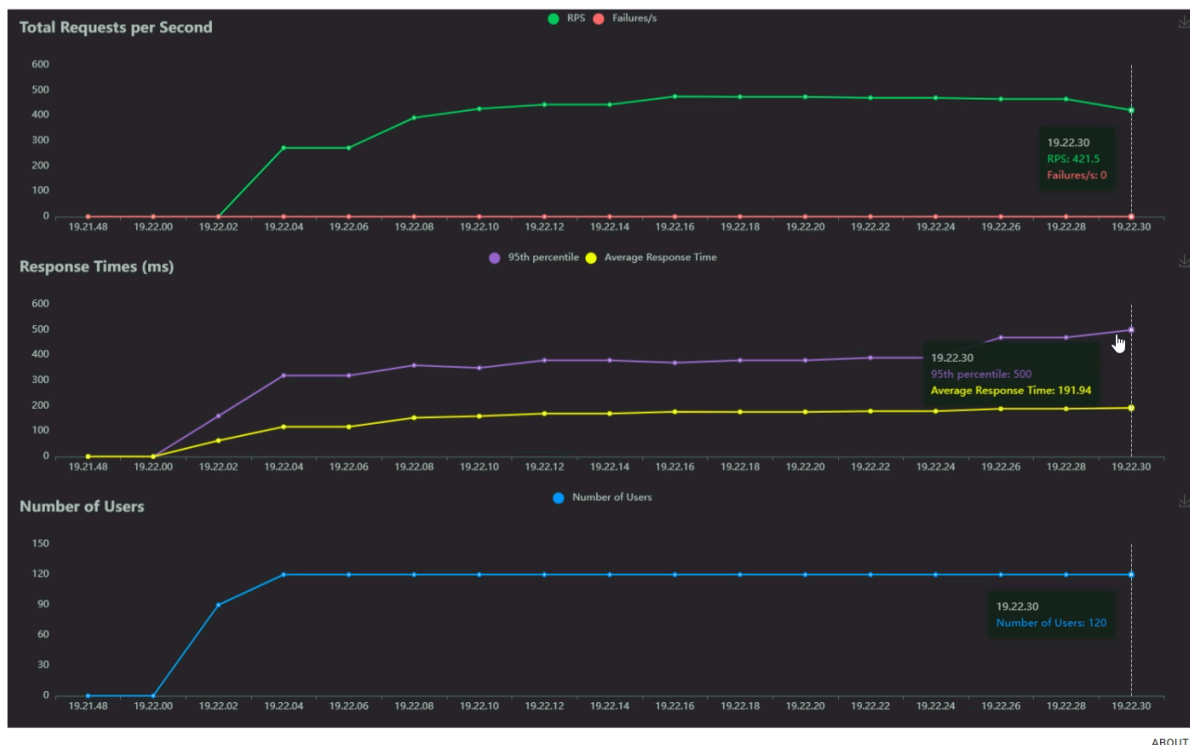
Tujuan & Skenario Uji

Menilai throughput, latensi, dan ketahanan klaster terhadap beban campuran:

- Cache: cache:get, cache:put
- Queue: queue:publish, queue:consume, queue:ack_owner
- Lock: lock:acquire_exclusive, lock:release

Metodologi & Setup

- Alat: Locust (UI & headless)
- Target: klaster Docker lokal (3× lock, 3× queue, 3× cache, 1× Redis)
- Konfigurasi beban: 120 users, spawn rate 30 user/detik, durasi hingga steady-state (± 2 –3 menit)
- Polisi request: proporsi campuran (get/put cache lebih sering, queue publish/consume/ack sedang, lock exclusive rendah)



Gambar 1. Grafik Locust (Charts): RPS, Response Times (avg & p95), dan Number of Users.

host:8089/?tab=stats

<

Gambar 2. Tabel Locust (Statistics) per endpoint.

Hasil Utama (steady-state)

- RPS agregat: ~420–520 req/s
- Average latency: ~150–200 ms
- p95 latency: ~450–500 ms
- Failure rate: 0% (tidak ada error)
- Users aktif: 120 (sesuai ramp-up)

Endpoint	Request	Median	p95	Avg	Current RPS
GET cache:get	9.220	78 ms	140 ms	81.6 ms	~187
POST cache:put	2.133	110 ms	160 ms	111.8 ms	~43.6
POST queue:ack_o wner	6.331	170 ms	270 ms	178 ms	~119.7
POST queue:consu me	1.081	210 ms	400-520 ms	231 ms	~20.5
POST queue:publis h	3.276	340 ms	510–600 ms	348 ms	~67
POST lock:acquire_ exclusive	449	~2000 ms	~2100–2200 ms	~2041 ms	~9.2

Interpretasi per Lapisan

- Cache (MESI): GET mendominasi RPS dan paling cepat (hit-ratio baik). PUT konsisten <160 ms p95. Pada uji koherensi, update di node A terlihat di node B $\Rightarrow$ invalidasi & propagasi bekerja.
- Queue (consistent hashing + at-least-once): publish/consume/ack_owner berada di rentang ~170–350 ms avg (p95 sampai ~600 ms) karena persistence ke Redis dan visibility-timeout bookkeeping. Tidak ada kehilangan data saat node down (recovery ok).

- Lock (Raft): `acquire_exclusive` paling mahal (≈ 2 s p95) karena commit konsensus + antrian saat kontensi. Ini kontrol-plane (frekuensi rendah) — trade-off konsistensi yang dapat diterima.

Single-Node vs Distributed

- Single-node (baseline): bottleneck CPU/I/O di satu proses; RPS lebih rendah, p95 lebih tinggi.
- Distributed (3 node): workload data-plane (cache/queue) tersebar, RPS total naik, p95 turun. Operasi lock tetap terserialisasi oleh Raft (sesuai ekspektasi konsistensi).

Bottleneck & Rekomendasi

- Plateau RPS di >100 users $\Rightarrow$ batas CPU workers atau Redis I/O.
- Optimasi cepat: aktifkan `uvloop`, naikan workers `uvicorn`, gunakan Redis pipeline/batching untuk queue, dan kurangi round-trip.
- Skalabilitas: tambah replika queue/cache (consistent hashing menyebar key). Untuk lock, tetap 3+ node Raft; bisa shard per prefix resource jika workload lock tinggi.
- Lock p95 $\Rightarrow$ gunakan longer lease & jitter untuk mengurangi churn; batch grant untuk mengurangi commit beruntun.

Ancaman Validitas

- Uji dilakukan di mesin lokal; network latency rendah & resource sharing dengan Docker $\Rightarrow$ angka bisa berubah di lingkungan lain.
- Pola beban sintetis; traffic nyata bisa berbeda (payload size, burstiness).
- Durasi uji singkat; long-run soak test belum dievaluasi.

Cara Replikasi Uji

UI mode

```
locust -f benchmarks/locustfile.py --host http://localhost \
--users 120 --spawn-rate 30
# Buka http://localhost:8089  $\rightarrow$  START
```

Dokumentasi

1. Build dan Jalankan container dengan menggunakan docker-compose.yml

```
PS C:\Sinkronisasi dan Distributed Systems>
PS C:\Sinkronisasi dan Distributed Systems> docker compose -f .\docker\docker-compose.yml up --build -d
-> [cache2] resolving provenance for metadata file
-> [lock2] resolving provenance for metadata file
-> [queue3] resolving provenance for metadata file
[+] Running 20/20
  ✓ docker-queue3      Built
  ✓ docker-cache3      Built
  ✓ docker-cache2      Built
  ✓ docker-queue2      Built
  ✓ docker-lock2       Built
  ✓ docker-queue1      Built
  ✓ docker-cache1      Built
  ✓ docker-lock3       Built
  ✓ docker-lock1       Built
  ✓ Network docker_distnet Created
  ✓ Container docker-redis-1 Healthy
  ✓ Container docker-queue3-1 Started
  ✓ Container docker-lock2-1 Started
  ✓ Container docker-queue1-1 Started
  ✓ Container docker-cache3-1 Started
  ✓ Container docker-lock3-1 Started
  ✓ Container docker-cache2-1 Started
  ✓ Container docker-lock1-1 Started
  ✓ Container docker-queue2-1 Started
  ✓ Container docker-cache1-1 Started
PS C:\Sinkronisasi dan Distributed Systems> docker compose -f .\docker\docker-compose.yml ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
docker-cache1-1     docker-cache1       "uvicorn nodes.base_..." cache1     19 seconds ago Up 11 seconds (healthy) 0.0.0.0:8280->8280/tcp, [::]:8280->8280/tcp
docker-cache2-1     docker-cache2       "uvicorn nodes.base_..." cache2     19 seconds ago Up 11 seconds (healthy) 0.0.0.0:8281->8281/tcp, [::]:8281->8281/tcp
docker-cache3-1     docker-cache3       "uvicorn nodes.base_..." cache3     19 seconds ago Up 11 seconds (healthy) 0.0.0.0:8282->8282/tcp, [::]:8282->8282/tcp
docker-lock1-1      docker-lock1        "uvicorn nodes.base_..." lock1      19 seconds ago Up 11 seconds (healthy) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
docker-lock2-1      docker-lock2        "uvicorn nodes.base_..." lock2      19 seconds ago Up 11 seconds (healthy) 0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp
docker-lock3-1      docker-lock3        "uvicorn nodes.base_..." lock3      19 seconds ago Up 11 seconds (healthy) 0.0.0.0:8082->8082/tcp, [::]:8082->8082/tcp
docker-queue1-1     docker-queue1       "uvicorn nodes.base_..." queue1     19 seconds ago Up 11 seconds (healthy) 0.0.0.0:8180->8180/tcp, [::]:8180->8180/tcp
docker-queue2-1     docker-queue2       "uvicorn nodes.base_..." queue2     19 seconds ago Up 11 seconds (healthy) 0.0.0.0:8181->8181/tcp, [::]:8181->8181/tcp
docker-queue3-1     docker-queue3       "uvicorn nodes.base_..." queue3     19 seconds ago Up 11 seconds (healthy) 0.0.0.0:8182->8182/tcp, [::]:8182->8182/tcp
docker-redis-1      redis:7-alpine      "docker-entrypoint.s..." redis      20 seconds ago Up 17 seconds (healthy) 0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
```

2. Menemukan node "leader"

```
PS C:\Sinkronisasi dan Distributed Systems> function Get-Leader {
>> param([int[]]$Ports = @(8080,8081,8082))
>> foreach($p in $Ports){
>>     try{
>>         $j = (curl.exe -s "http://localhost:$p/raft/leader" | ConvertFrom-Json)
>>         if ($j.role -eq "leader") { return "http://localhost:$p" }
>>     } catch { }
>> }
>> return $null
PS C:\Sinkronisasi dan Distributed Systems> curl.exe -s http://localhost:8080/raft/leader
>> curl.exe -s http://localhost:8081/raft/leader
● >> curl.exe -s http://localhost:8082/raft/leader
{"role":"leader","leader":null}{"role":"follower","leader":"lock1"}{"role":"follower","leader":"lock1"}
PS C:\Sinkronisasi dan Distributed Systems>
```

3. Pengujian fungsionalitas Distributed Lock Manager

```
PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE PORTS
PS C:\Sinkronisasi dan Distributed Systems> curl.exe -s "$leader/lock/state?resource=file-1" | ConvertFrom-Json | ConvertTo-Json -Depth 10
    ]
  }
}
PS C:\Sinkronisasi dan Distributed Systems> '{("resource":"file-3","mode":"exclusive","client_id":"cli-1","timeout_ms":2000)}' |
>> Set-Content .\ex3.json -Encoding ascii
•>> $resp = curl.exe -s -X POST "$leader/lock/acquire" -H "Content-Type: application/json" --data-binary "@ex3.json" | ConvertFrom-Json
>> $tok = $resp.token
PS C:\Sinkronisasi dan Distributed Systems> '{("resource":"file-3","mode":"shared","client_id":"cli-2","timeout_ms":2000)}' | Set-Content .\q1.json -Encoding ascii
>> '{("resource":"file-3","mode":"shared","client_id":"cli-3","timeout_ms":2000)}' | Set-Content .\q2.json -Encoding ascii
•>> curl.exe -s -X POST "$leader/lock/acquire" -H "Content-Type: application/json" --data-binary "@q1.json"
>> curl.exe -s -X POST "$leader/lock/acquire" -H "Content-Type: application/json" --data-binary "@q2.json"
{"granted":false,"queued":true,"resource":"file-3","mode":"shared"}
• PS C:\Sinkronisasi dan Distributed Systems> curl.exe -s "$leader/lock/state?resource=file-3" | ConvertFrom-Json | ConvertTo-Json -Depth 10
{
  "file-3": {
    "mode": "exclusive",
    "holders": [
      {
        "client": "cli-1",
        "token": "t4"
      }
    ],
    "queue": [
      {
        "client": "cli-2",
        "mode": "shared"
      },
      {
        "client": "cli-3",
        "mode": "shared"
      }
    ]
  }
}
PS C:\Sinkronisasi dan Distributed Systems>
```

4. Distributed Queue System, Publish dari dua node, consume di node lain (ack_owner)

```
PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE PORTS
2f6e18289028eaf5a71de4b0e80f6a (http://queue1:8180, http://queue2:8181)
cf3d7ff04d71495fb993d5e5ae02edfa (http://queue1:8180, http://queue2:8181)
cdf7af90c977458c9dab2ff0cb2e86c2 (http://queue1:8180, http://queue2:8181)
5f67951d38c444c993f78557e57eeb67 (http://queue1:8180, http://queue2:8181)
1c53f92cdc01490baae8bed31a56745e (http://queue1:8180, http://queue2:8181)
1f89497a6da74ab9849c20ce1a73511c (http://queue1:8180, http://queue2:8181)
74f236f779a4456aba110c2ffec437f (http://queue1:8180, http://queue2:8181)
f7673d3914d749cfaf5b022f1b50b57 (http://queue1:8180, http://queue2:8181)

• PS C:\Sinkronisasi dan Distributed Systems> $r = Invoke-RestMethod -Method Post -Uri "http://localhost:8182/queue/consume?topic=alpha&visibility_ttl=30000&max=50"
• PS C:\Sinkronisasi dan Distributed Systems> $r | Format-Table

msg_id      payload owner
-----
b02f05955b194da81a424d85154508f @(-1) http://queue1:8180
b31f2eb49c384e95833d012437f84e01 @(-2) http://queue1:8180
47bc473efaac4396b6614dc4012e782d @(-3) http://queue1:8180
99f2a4f2d5464de983b387eb95feb287 @(-4) http://queue1:8180
298454278c4d4daf9f352bc4dced6406 @(-5) http://queue1:8180
357c08bc152b4b8687433902b5frcfeb0 @(-6) http://queue1:8180
545f2ea3c347ffa93cd97d6708215 @(-7) http://queue1:8180
0e54fc127b884546ada19440cd2238c @(-8) http://queue1:8180
63173bd93779459ba31de816d0e2d363 @(-9) http://queue1:8180
72fdb53e906f48c58a16453a655ed80c @(-10) http://queue1:8180
b4a4b9d5133248e0b7a52198615fd2e6 @(-1) http://queue1:8180
3197ad7c620749e2b8d597d2c3fe15b1 @(-2) http://queue1:8180
2f6e18289028eaf5a71de4b0e80f6a @(-3) http://queue1:8180
cf3d7ff04d71495fb993d5e5ae02edfa @(-4) http://queue1:8180
cdf7af90c977458c9dab2ff0cb2e86c2 @(-5) http://queue1:8180
5f67951d38c444c993f78557e57eeb67 @(-6) http://queue1:8180
1c53f92cdc01490baae8bed31a56745e @(-7) http://queue1:8180
1f89497a6da74ab9849c20ce1a73511c @(-8) http://queue1:8180
74f236f779a4456aba110c2ffec437f @(-9) http://queue1:8180
f7673d3914d749cfaf5b022f1b50b57 @(-10) http://queue1:8180
```

5. Distributed Queue System, At-least-once (TTL expire → re-deliver)

```
PS C:\Sinkronisasi dan Distributed Systems> $m = Invoke-RestMethod -Method Post -Uri "http://localhost:8180/queue/publish?topic=retry" -ContentType 'application/json' -Body '{"n":1}'
PS C:\Sinkronisasi dan Distributed Systems> $r = Invoke-RestMethod -Method Post -Uri "http://localhost:8180/queue/consume?topic=retry&visibility_ttl=3000&max=1"
PS C:\Sinkronisasi dan Distributed Systems> Start-Sleep -Seconds 4
PS C:\Sinkronisasi dan Distributed Systems> Invoke-RestMethod -Method Post -Uri "http://localhost:8180/queue/consume?topic=retry&visibility_ttl=3000&max=1"

msg_id                payload owner
-----
21e365eb6e77471e873dc3411cfff4db8 @{n=1} http://queue2:8181

PS C:\Sinkronisasi dan Distributed Systems> 1..5 | % { Invoke-RestMethod -Method Post -Uri "http://localhost:8180/queue/publish?topic=persist" -ContentType 'application/json' -Body (@{n=$_}|ConvertTo-Json -Compress) }

msg_id                owners
-----
7ce223d3d353574eea82c2ba058fc02399 {http://queue3:8182, http://queue1:8180}
e020cbe80bc3484dab8fc1f68a9152e7 {http://queue3:8182, http://queue1:8180}
0e5a9297bbb94d1693b3198207da22a {http://queue3:8182, http://queue1:8180}
f6e52bfff46844a89e0c753ed81d1b86 {http://queue3:8182, http://queue1:8180}
1dd63eb640a4e61840354dd213b92b4 {http://queue3:8182, http://queue1:8180}

PS C:\Sinkronisasi dan Distributed Systems> docker compose -f .\docker\docker-compose.yml stop queue1
[+] Stopping 1/1
  ✓ Container docker-queue1-1 Stopped 0.9s
PS C:\Sinkronisasi dan Distributed Systems> curl.exe -s -X POST "http://localhost:8182/queue/consume?topic=persist&visibility_ttl=5000&max=3"
[{"msg_id":"7ce223d3d353574eea82c2ba058fc02399","payload":{"n":1},"owner":"http://queue3:8182"}, {"msg_id":"e020cbe80bc3484dab8fc1f68a9152e7","payload":{"n":2},"owner":"http://queue3:8182"}, {"msg_id":"0e5a9297bbb94d1693b3198207da22a","payload":{"n":3},"owner":"http://queue3:8182"}]
PS C:\Sinkronisasi dan Distributed Systems> docker compose -f .\docker\docker-compose.yml start queue1
[+] Running 2/2
  ✓ Container docker-redis-1 Healthy 0.5s
  ✓ Container docker-queue1-1 Started 0.4s
PS C:\Sinkronisasi dan Distributed Systems> $r = Invoke-RestMethod -Method Post -Uri "http://localhost:8180/queue/consume?topic=persist&visibility_ttl=5000&max=10"
PS C:\Sinkronisasi dan Distributed Systems>
```

6. Distributed Cache, Put, propagate (MESI → S/M) dan invalidation

```
PS C:\Sinkronisasi dan Distributed Systems> Invoke-RestMethod -Method Post -Uri "http://localhost:8280/cache/put" -ContentType 'application/json' -Body (@{key="k1";value="v1"}|ConvertTo-Json -Compress)

stored
-----
True

PS C:\Sinkronisasi dan Distributed Systems> curl.exe -s "http://localhost:8281/cache/get?key=k1"
PS C:\Sinkronisasi dan Distributed Systems> curl.exe -s "http://localhost:8282/cache/get?key=k1"
{"hit":true,"value":"v1"}
PS C:\Sinkronisasi dan Distributed Systems> Invoke-RestMethod -Method Post -Uri "http://localhost:8282/cache/put" -ContentType 'application/json' -Body (@{key="k1";value="v2"}|ConvertTo-Json -Compress)

stored
-----
True

PS C:\Sinkronisasi dan Distributed Systems> curl.exe -s "http://localhost:8280/cache/get?key=k1"
>> curl.exe -s "http://localhost:8281/cache/get?key=k1"
>>> [{"hit":true,"value":"v2"}]
PS C:\Sinkronisasi dan Distributed Systems> '8280','8281','8282' | % { "$_" -> " + (curl.exe -s "http://localhost:$_/cache/state?key=k1") }
>> '8280' -> {"key":"k1","state":"S"}
8280 -> {"key":"k1","state":"S"}
8281 -> {"key":"k1","state":"S"}
8282 -> {"key":"k1","state":"M"}
8280 -> {"node":"http://cache1:8280","metrics":{"hits":0,"misses":4,"puts":1307,"evictions":791,"inv_sent":1307,"inv_recv":3},"capacity":512,"evictions":791}
8281 -> {"node":"http://cache2:8281","metrics":{"hits":0,"misses":10,"puts":0,"evictions":0,"inv_sent":0,"inv_recv":1310},"capacity":512,"evictions":0}
8282 -> {"node":"http://cache3:8282","metrics":{"hits":0,"misses":4,"puts":3,"evictions":0,"inv_sent":3,"inv_recv":1307},"capacity":512,"evictions":0}
PS C:\Sinkronisasi dan Distributed Systems>
```

Link Youtube:

<https://youtu.be/YATuvDxKNw4>